ResolveAI

A helper that finds out why something broke, with proof

> **In one line:** when a live system breaks, ResolveAI collects the clues, asks Claude (an AI) for the most likely causes with proof, and lets an engineer check the answer and write the report.

1. The problem

Companies run software services that customers use all day. Sometimes a service breaks: pages are slow, payments fail, or errors appear. We call this an **incident**.

When an incident happens, an engineer must find out why it happened, fix it, and later write a report about it. Today this is slow and hard:

- The clues are in many places: logs (messages the software writes), deploys (new versions that were released), and metrics (charts of speed and errors).
- The engineer has to open many tools and read a lot of text, often at night and under pressure.
- The same kind of problem may have happened before, but nobody remembers how it was fixed.
- After the fix, someone must write a report (an RCA, root cause analysis) by hand. Notes get lost.
- AI tools can read fast, but they sometimes make things up, like a number or a cause that has no proof.
- Logs can hold secrets (passwords, tokens, emails). These must not be sent to an AI.

What we want

- Find the likely cause in minutes, not hours.
- Every answer must show its proof, so the engineer can trust it or reject it.
- A person always makes the final decision. The AI never changes the live system.
- The final report is mostly written for you, and you only edit it.

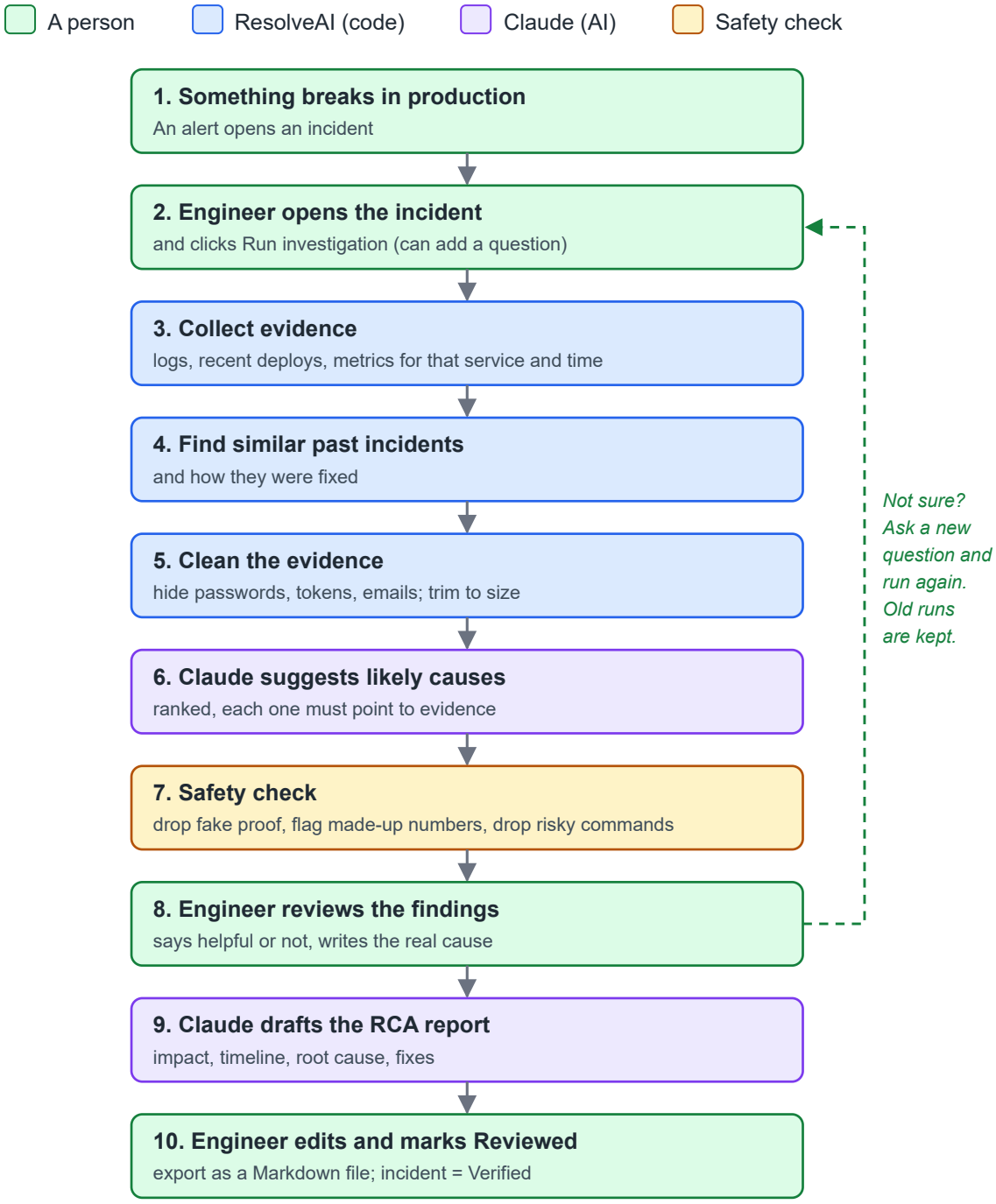
2. The solution in simple words

ResolveAI is a web app. The engineer opens an incident and clicks **Run investigation**. ResolveAI does the reading work, and Claude does the thinking work. Then a person checks it. These are the steps:

Step	What happens	Who
1. Pick the incident	The engineer opens the incident from the list. They can also type a question, like "Is the new release the cause?"	Person
2. Collect evidence	ResolveAI gathers the logs, deploys and metrics for that service, from just before the problem started until shortly after it was detected.	ResolveAI
3. Look at the past	It searches old incidents for similar problems and how they were fixed.	ResolveAI
4. Clean the evidence	It hides secrets (passwords, tokens, emails) and cuts the text to a safe size. Each clue gets an ID, like log:… or mcp:3.	ResolveAI
5. Ask Claude	Claude reads the clean evidence and lists the likely causes, best first. Each cause must name the clue IDs that prove it, and say what is still unknown.	Claude (AI)
6. Check the answer	ResolveAI removes any proof ID that does not exist, flags numbers (like "35%") that are not in the evidence, and removes risky commands.	ResolveAI
7. Show the result	The engineer sees the causes, the proof for each, and a "How this was produced" panel with every step and its time.	ResolveAI
8. Engineer decides	The engineer marks each cause as helpful or not and writes the real root cause. They can run it again; old runs are kept.	Person
9. Draft the report	Claude writes a first draft of the RCA report: impact, timeline, root cause, fixes and how to stop it happening again.	Claude (AI)
10. Finish the report	The engineer edits the draft, marks it Reviewed and downloads it. The incident now shows Verified in the list.	Person

3. Workflow diagram

The boxes go from top to bottom. The colour shows who does each step.



4. Safety rules

- **No proof, no claim.** Every cause must point to real evidence. Fake proof is removed.
- **Numbers are checked.** If Claude writes a number that is not in the evidence, it is flagged so the engineer sees it.
- **Secrets stay private.** Passwords, tokens and emails are hidden before anything goes to Claude.
- **The AI only advises.** It never runs commands, restarts servers or rolls back releases.
- **A person has the last word.** An incident is only Verified after a person confirms the cause and reviews the report.
- **Cost is controlled.** By default the app uses saved (recorded) answers, which cost nothing. Live Claude must be switched on in the settings, and has a daily limit.

5. How to run it

Get the code: run `git clone https://github.com/ratnkumarsingh/resolveai-agentic.git`, then `cd resolveai-agentic`.

On a developer computer: install the .NET 10 SDK, run `dotnet run --project src/ResolveAI.App`, and open http://localhost:8787.

In a Docker container on your own computer: run `docker compose up -d --build` and open http://localhost:8787. The data is kept when the container restarts.

To use live Claude: put your API key in the `.env` file. Then either set `LLM_MODE=live`, or add an access code to the file and type it on the Settings page.

6. Words to know

Incident	A time when a live service is broken or slow.
Log	A line of text the software writes about what it is doing, such as an error.
Deploy	Releasing a new version of the software.
Metric	A number measured over time, like error rate or response time.
Evidence	The clues (logs, deploys, metrics) that ResolveAI collects.
Hypothesis	A possible cause that still needs to be checked.
Root cause	The real reason the incident happened.
RCA	Root cause analysis: the report written after an incident.
Recorded mode	The app uses saved answers instead of calling Claude. Free and always the same.
Live mode	The app calls Claude for real. Needs an API key and costs a little money.